

Documentación Técnica

Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

Estudiante	Henry Alexander García Montúfar
Carnet	201407049
Proyecto	Primer Proyecto — Apache Cassandra
Repositorio	https://github.com/Halex07/BD2_2S26_Proyectos_201407049.git
Fecha	Septiembre de 2026

1. Marco formativo

1.1 Valor: Responsabilidad

Se asumió la responsabilidad de diseñar e implementar correctamente el modelo de datos distribuido en Apache Cassandra, garantizando la disponibilidad, consistencia y eficiencia del sistema de reservas desarrollado. Esto se refleja en las pruebas de integridad tras la carga masiva y en las pruebas de tolerancia a fallos documentadas en la sección 7.

1.2 Competencias

General: Construcción de una solución integral de forma autónoma, documentando cada decisión de diseño para que sea reproducible por un tercero.

Específica: Implementación de técnicas de modelado de datos distribuidos en Apache Cassandra (Query-Driven Modeling) para optimizar el almacenamiento y la consulta de información, garantizando escalabilidad, alta disponibilidad y eficiencia ante grandes volúmenes de datos.

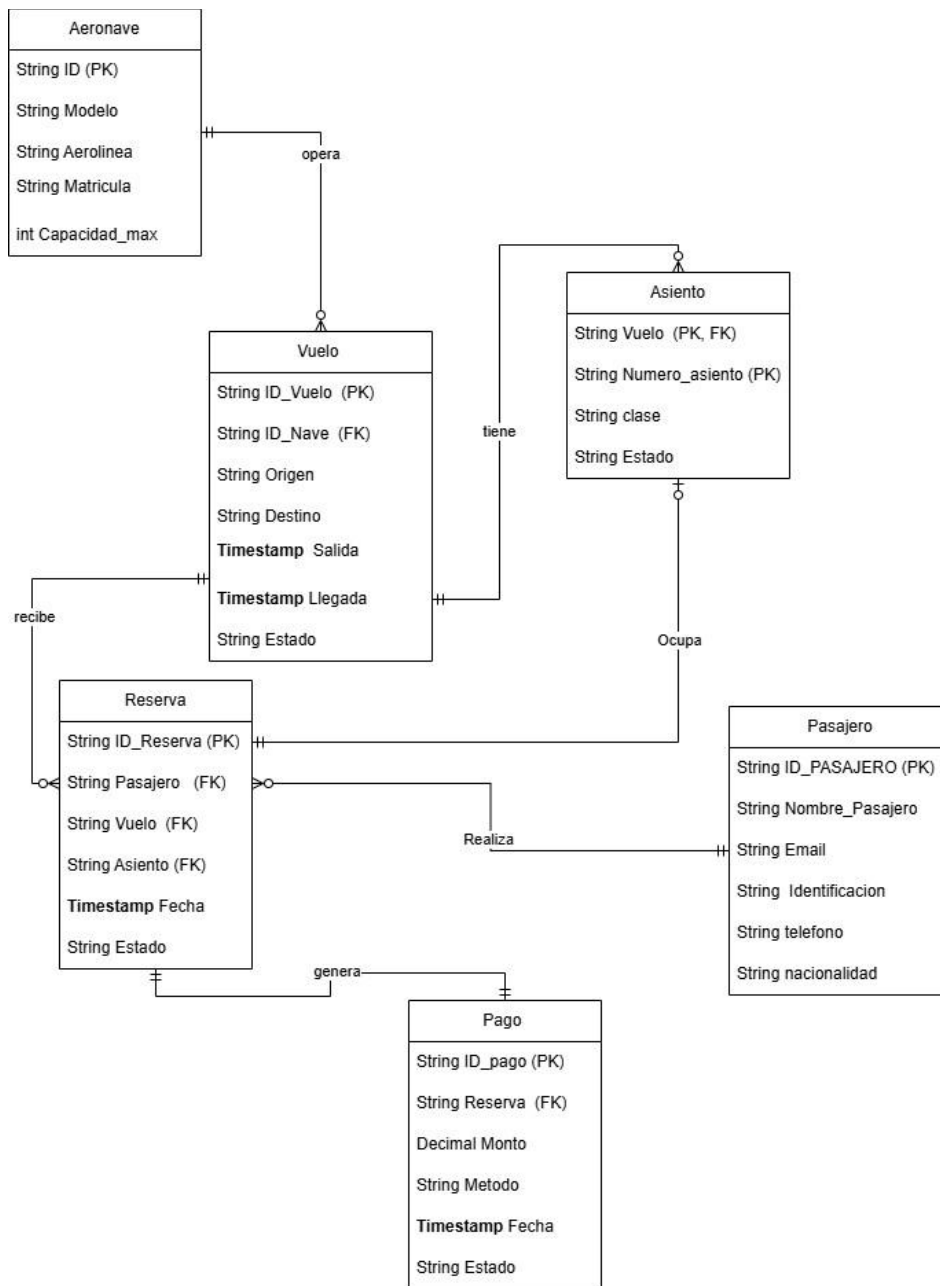
2. Resumen ejecutivo

El proyecto implementa una base de datos distribuida en Apache Cassandra 4.1 para la gestión de reservas y boletos de una aerolínea. El diseño sigue el principio de *Query-Driven Modeling*: primero se identificaron las 5 consultas de negocio requeridas y, a partir de ellas, se definieron 12 tablas especializadas con claves de partición y clustering propias, evitando por completo JOINS y `ALLOW FILTERING`.

El clúster se desplegó con 3 nodos en Docker, con `NetworkTopologyStrategy` y Replication Factor 3. Se cargaron 100,000 reservas (con 20,000 pasajeros y 800 vuelos) mediante Batch Writes, verificando la integridad de cada tabla y de los contadores contra los datos denormalizados. Se implementaron y midieron las 5 consultas requeridas, se ejecutaron pruebas de tolerancia a fallos con distintos Consistency Levels, y se demostraron dos reglas de negocio propias de un entorno distribuido: control de concurrencia con Lightweight Transactions (LWT) y expiración automática de reservas pendientes con TTL.

3. Modelo entidad-relación conceptual

El modelo conceptual representa la lógica de negocio del dominio, previa a su transformación al modelo lógico de Cassandra. Las 6 entidades principales son Aeronave, Vuelo, Asiento, Pasajero, Reserva y Pago.



Cardinalidades: Aeronave 1:N Vuelo · Vuelo 1:N Asiento · Pasajero 1:N Reserva · Vuelo 1:N Reserva · Reserva 1:0..1 Asiento (un asiento disponible aún no tiene reserva) · Reserva 1:1 Pago.

4. Modelo lógico basado en queries (Query-Driven Modeling)

En Cassandra el modelo no se normaliza: se diseña a partir de las consultas que el sistema debe responder. Cada consulta de negocio tiene su propia tabla, con los datos denormalizados que necesita, de modo que se resuelve leyendo una sola partición y sin JOINS.

4.1 Keyspace y estrategia de replicación

```
CREATE KEYSPACE aerolinea WITH replication = {'class': 'NetworkTopologyStrategy', 'dc1': 3};
```

Se eligió **NetworkTopologyStrategy** sobre SimpleStrategy porque es topology-aware: reconoce datacenters y racks, lo que la hace la estrategia recomendada para cualquier despliegue que pueda crecer a más de un datacenter, incluso si hoy el clúster vive en uno solo (dc1). Con Replication Factor 3 en un clúster de 3 nodos, cada fila tiene una copia completa en cada nodo, lo que permite:

- CL=ONE: tolera hasta 2 nodos caídos (1 de 3 réplicas basta).

```
root@171c27e71b9a:/app# python python/pruebas_consistencia.py --escenario 1_nodo_caido
(aviso) cass3 no resuelve: se asume caido
Escenario: 1_nodo_caido | 100 operaciones por CL y tipo

ONE  lectura  exitos 100/100 avg 1.59 ms p95 2.22 ms
ONE  escritura exitos 100/100 avg 1.05 ms p95 1.81 ms
QUORUM lectura  exitos 100/100 avg 2.58 ms p95 3.52 ms
QUORUM escritura exitos 100/100 avg 1.47 ms p95 2.09 ms
ALL  lectura  exitos 0/100 avg - ms p95 - ms Unavailable=100
ALL  escritura exitos 0/100 avg - ms p95 - ms Unavailable=100

Guardado en resultados/consistencia.csv. Siguiendo escenario, o --resumen al terminar.
root@171c27e71b9a:/app#
```

- CL=QUORUM: tolera 1 nodo caído (necesita 2 de 3 réplicas).

```
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker stop cass2
cass2
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load      Tokens    Owns (effective)  Host ID                               Rack
DN  172.18.0.4    34.64 MiB  16        100.0%            14873b2e-ec80-46b2-9e5d-d7b731000845  rack1
DN  172.18.0.3    34.65 MiB  16        100.0%            9dbe660f-68b9-488c-84ce-f0525e13cfb5  rack1
UN  172.18.0.2    34.6 MiB   16        100.0%            69cb9021-8ddd-4c9b-8cf2-539168bcac19  rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1>
```

- CL=ALL: no tolera ningún nodo caído (requiere las 3 réplicas).

```
try docker debug for seamless, persistent debugging tools in any container or image → docker debug cass1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker stop cass3
cass3
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load      Tokens    Owns (effective)  Host ID                               Rack
DN  172.18.0.4    34.64 MiB  16        100.0%            14873b2e-ec80-46b2-9e5d-d7b731000845  rack1
UN  172.18.0.3    34.65 MiB  16        100.0%            9dbe660f-68b9-488c-84ce-f0525e13cfb5  rack1
UN  172.18.0.2    34.6 MiB   16        100.0%            69cb9021-8ddd-4c9b-8cf2-539168bcac19  rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1>
```

4.2 Tabla que responde cada consulta del negocio

Consulta	Tabla	Primary Key	Estrategia
Q1. Disponibilidad de asientos por clase	asientos_contadores_por_vuelo	((codigo_vuelo), clase)	Tabla de contadores (counter), actualizada en cada reserva/cancelación en lugar de contar filas en tiempo de consulta.
Q2. Historial cronológico de un pasajero	historial_por_pasajero	((pasajero_id), fecha_salida, reserva_id)	Clustering key por fecha con ORDER BY y comparadores de rango; denormaliza reserva + vuelo + pago en una sola fila.
Q3. Manifiesto de vuelo enriquecido	manifiesto_por_vuelo	((codigo_vuelo), fila, letra)	Clustering (fila, letra) da el orden por asiento; denormaliza pasajero, reserva y pago en una sola tabla, sin JOIN.
Q4. % de ocupación por ruta y rango de fechas	ocupacion_por_ruta_mes	((ruta, mes), fecha_salida, codigo_vuelo, capacidad)	Bucketing de la partición por (ruta, mes) para evitar particiones gigantes; contador de reservas confirmadas junto con la capacidad.
Q5. Top N de vuelos por ingresos	top_vuelos_por_ingresos	((periodo), total_centavos, codigo_vuelo)	Clustering key en orden DESC sobre el monto acumulado; LIMIT N se resuelve sin ordenar ni filtrar en el cliente.

4.3 Tablas de apoyo (una por entidad y por búsqueda por identificador)

Tabla	Primary Key	Uso
pasajeros_por_id	((pasajero_id))	Datos de un pasajero por su id.
Tabla	Primary Key	Uso
aeronaves_por_id	((aeronave_id))	Datos de una aeronave por su id.
vuelos_por_codigo	((codigo_vuelo))	Datos de un vuelo, incluida la ruta y capacidad denormalizadas.
asientos_por_vuelo	((codigo_vuelo), fila, letra)	Estado de cada asiento de un vuelo; base de la regla de negocio de asiento único (LWT, sección 8.1).
reservas_por_id	((reserva_id))	Una reserva por id; base para cancelar/modificar.

pagos_por_reserva	((reserva_id))	El pago de una reserva (relación 1:1).
ingresos_por_vuelo	((codigo_vuelo))	Contador auxiliar del ingreso acumulado por vuelo, usado para recalcular la tabla de ranking (Q5) cuando cambia un pago.

4.4 Decisiones de diseño relevantes

- **Asiento con fila (int) + letra (text):** se evitó un identificador de asiento como texto único (p. ej. "12A") porque un ORDER BY ordenaría "10A" antes que "2A" al comparar cadenas. Con fila como entero, el clustering ordena numéricamente.
- **Bucketing en Q4 (ruta + mes):** sin el bucket por mes, la partición de una ruta muy transitada crecería sin límite con el tiempo. Bucketear por mes acota el tamaño de cada partición.
- **Contadores no admiten TTL ni columnas normales** en la misma tabla: por eso capacidad y confirmadas conviven como columnas normal/counter distintas en la misma fila (capacidad es parte de la clustering key), y por eso el ranking de ingresos (Q5) usa dos tablas: un contador auxiliar (ingresos_por_vuelo) y una tabla de ranking que se reescribe cuando el contador cambia.
- **Batch LOGGED por reserva:** cada reserva se escribe en 3 o 4 tablas denormalizadas (reservas_por_id, pagos_por_reserva, historial_por_pasajero y, si no está cancelada, manifiesto_por_vuelo). Se agrupan en un único batch logged para que las copias queden consistentes entre sí.

5. Configuración del clúster

El clúster se desplegó con Docker Compose, con 3 contenedores basados en la imagen oficial `cassandra:4.1`, en la misma red Docker (`cassnet`) y con arranque escalonado (cada nodo espera a que el anterior esté *healthy* antes de unirse).

Parámetro	Valor
Nodos	3 (cass1, cass2, cass3)
Datacenter / Rack	dc1 / rack1 (GossipingPropertyFileSnitch)
Estrategia de replicación	NetworkTopologyStrategy
Replication Factor	3
Consistency Level por defecto (aplicación)	QUORUM
Heap por nodo	512 MB (entorno de laboratorio en Docker)

Verificación del estado del clúster con los 3 nodos activos (`nodetool status`):

```
Datacenter: dc1
Status=Up/Down |/ State=Normal/Leaving/Joining/Moving
-- Address Load Tokens Owns (effective) Host ID Rack
UN 172.18.0.4 75.2 KiB 16 76.0% 14873b2e-ec80-46b2-9e5d-d7b731000845 rack1
UN 172.18.0.3 75.2 KiB 16 59.3% 9dbe660f-68b9-488c-84ce-f0525e13cfb5 rack1
UN 172.18.0.2 109.4 KiB 16 64.7% 69cb9021-8ddd-4c9b-8cf2-539168bcac19 rack1
```

Los 3 nodos aparecen con estado **UN** (Up/Normal). Con RF=3 en un clúster de 3 nodos, la columna *Owns* tiende al 100% en cada nodo porque cada uno contiene una réplica completa de todos los datos.

6. Carga masiva de datos

Se generaron datos sintéticos reproducibles (semilla fija) con la librería *Faker* y se cargaron con el driver oficial de Python para Cassandra (*cassandra-driver*).

Entidad	Cantidad
Pasajeros	20,000
Aeronaves	12
Vuelos	800 (entre octubre de 2026 y marzo de 2027)
Asientos	114,000
Reservas	100,000 (80,003 confirmadas · 12,037 pendientes · 7,960 canceladas)
Pagos	100,000

6.1 Estrategia de escritura

- **Un BATCH LOGGED por reserva:** agrupa las escrituras en las tablas denormalizadas que representan la misma reserva, garantizando que queden consistentes entre sí.
- **Contadores acumulados en memoria y aplicados al final** con BATCH COUNTER: se transforman 100,000 incrementos en aproximadamente 800 escrituras de contador (una por vuelo).
- **Ejecución concurrente** con *execute_concurrent* (128 operaciones simultáneas) para aprovechar los 3 nodos en paralelo.

6.2 Validación de integridad tras la carga

Tabla	Esperado	Real	Estado
pasajeros_por_id	20,000	20,000	OK
vuelos_por_codigo	800	800	OK
asientos_por_vuelo	114,000	114,000	OK
reservas_por_id	100,000	100,000	OK
pagos_por_reserva	100,000	100,000	OK
historial_por_pasajero	100,000	100,000	OK

manifiesto_por_vuelo	92,040	92,040	OK
----------------------	--------	--------	----

manifiesto_por_vuelo tiene menos filas que reservas_por_id porque las reservas *canceladas* no ocupan asiento ni aparecen en el manifiesto (100,000 – 7,960 canceladas = 92,040). Adicionalmente, se verificó en una muestra de 10 vuelos que la suma de los contadores de asientos_contadores_por_vuelo (Q1) coincide exactamente con el conteo real de filas en manifiesto_por_vuelo para cada vuelo, confirmando que los contadores no quedaron desincronizados de los datos denormalizados.

7. Consultas CQL implementadas

Las 5 consultas se probaron sobre el vuelo GT0476-20261204 (ruta MIA-LAX, capacidad 120) y se ejecutaron con TRACING ON para documentar su plan de ejecución. Ninguna consulta utiliza ALLOW FILTERING.

Q1 — Disponibilidad de asientos por clase

Clase	Disponibles	Ocupados
economica	14	70
ejecutiva	3	21
primera	4	8

El trace confirma `Executing single-partition query` sobre `asientos_contadores_por_vuelo`: se lee una única partición (`codigo_vuelo`) y los 3 valores por clase provienen de memtable (los contadores recién escritos aún no se habían compactado a disco).

```
qlsh:aerolinea> -- Q1
qlsh:aerolinea> SELECT clase, disponibles, ocupados
... FROM asientos_contadores_por_vuelo
... WHERE codigo_vuelo = 'GT0476-20261204';

clase | disponibles | ocupados
-----|-----|-----
economica | 14 | 70
ejecutiva | 3 | 21
primera | 4 | 8

(3 rows)

Tracing session: 0fecb589-b548-11f1-ba91-538c2587f0f1

activity | timestamp | source | source_elapsed | client
-----|-----|-----|-----|-----
Execute CQL3 query | 2026-09-20 23:07:33.467000 | 172.18.0.2 | 0 | 127.0.0.1
Parsing SELECT clase, disponibles, ocupados FROM asientos_contadores_por_vuelo WHERE codigo_vuelo = 'GT0476-20261204'; [Native-Transport-Requests-1] | 2026-09-20 23:07:33.475000 | 172.18.0.2 | 9286 | 127.0.0.1
Preparing statement [Native-Transport-Requests-1] | 2026-09-20 23:07:33.475000 | 172.18.0.2 | 9790 | 127.0.0.1
reading digest from /172.18.0.4:7000 [Native-Transport-Requests-1] | 2026-09-20 23:07:33.476000 | 172.18.0.2 | 10462 | 127.0.0.1
Executing single-partition query on asientos_contadores_por_vuelo [ReadStage-2] | 2026-09-20 23:07:33.476001 | 172.18.0.2 | 10496 | 127.0.0.1
Acquiring sstable references [ReadStage-2] | 2026-09-20 23:07:33.476002 | 172.18.0.2 | 10538 | 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-2] | 2026-09-20 23:07:33.476003 | 172.18.0.2 | 10661 | 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-2] | 2026-09-20 23:07:33.476004 | 172.18.0.2 | 10804 | 127.0.0.1
Read 3 live rows and 0 tombstone cells [ReadStage-2] | 2026-09-20 23:07:33.477000 | 172.18.0.2 | 10887 | 127.0.0.1
Sending READ_REQ message to /172.18.0.4:7000 message size 139 bytes [Messaging-EventLoop-3-9] | 2026-09-20 23:07:33.477001 | 172.18.0.2 | 11137 | 127.0.0.1
speculating read retry on Full/172.18.0.3:7000 (-13078026581242231501-99251617246320886148) [Native-Transport-Requests-1] | 2026-09-20 23:07:33.478000 | 172.18.0.2 | 12059 | 127.0.0.1
Sending READ_REQ message to /172.18.0.3:7000 message size 139 bytes [Messaging-EventLoop-3-2] | 2026-09-20 23:07:33.478001 | 172.18.0.2 | 12383 | 127.0.0.1
READ_REQ message received from /172.18.0.2:7000 [Messaging-EventLoop-3-14] | 2026-09-20 23:07:33.480000 | 172.18.0.4 | 439 | 127.0.0.1
READ_REQ message received from /172.18.0.2:7000 [Messaging-EventLoop-3-15] | 2026-09-20 23:07:33.481000 | 172.18.0.3 | 415 | 127.0.0.1
Executing single-partition query on asientos_contadores_por_vuelo [ReadStage-1] | 2026-09-20 23:07:33.484000 | 172.18.0.4 | 4625 | 127.0.0.1
Acquiring sstable references [ReadStage-1] | 2026-09-20 23:07:33.484001 | 172.18.0.4 | 4728 | 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-1] | 2026-09-20 23:07:33.484002 | 172.18.0.4 | 4931 | 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-1] | 2026-09-20 23:07:33.485000 | 172.18.0.4 | 5307 | 127.0.0.1
Read 3 live rows and 0 tombstone cells [ReadStage-1] | 2026-09-20 23:07:33.485001 | 172.18.0.4 | 5459 | 127.0.0.1
Enqueuing response to /172.18.0.2:7000 [ReadStage-1] | 2026-09-20 23:07:33.485002 | 172.18.0.4 | 5497 | 127.0.0.1
Sending READ_RSP message to cass1/172.18.0.2:7000 message size 51 bytes [Messaging-EventLoop-3-1] | 2026-09-20 23:07:33.485003 | 172.18.0.4 | 6005 | 127.0.0.1
READ_RSP message received from /172.18.0.4:7000 [Messaging-EventLoop-3-4] | 2026-09-20 23:07:33.486000 | 172.18.0.2 | 20352 | 127.0.0.1
Executing single-partition query on asientos_contadores_por_vuelo [ReadStage-3] | 2026-09-20 23:07:33.486000 | 172.18.0.3 | 5364 | 127.0.0.1
Acquiring sstable references [ReadStage-3] | 2026-09-20 23:07:33.486001 | 172.18.0.3 | 5559 | 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-3] | 2026-09-20 23:07:33.486002 | 172.18.0.3 | 5912 | 127.0.0.1
Processing response from /172.18.0.4:7000 [RequestResponseStage-2] | 2026-09-20 23:07:33.487000 | 172.18.0.2 | 21183 | 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-3] | 2026-09-20 23:07:33.487000 | 172.18.0.3 | 6298 | 127.0.0.1
Read 3 live rows and 0 tombstone cells [ReadStage-3] | 2026-09-20 23:07:33.487001 | 172.18.0.3 | 6552 | 127.0.0.1
Enqueuing response to /172.18.0.2:7000 [ReadStage-3] | 2026-09-20 23:07:33.487002 | 172.18.0.3 | 6706 | 127.0.0.1
Sending READ_RSP message to cass1/172.18.0.2:7000 message size 339 bytes [Messaging-EventLoop-3-1] | 2026-09-20 23:07:33.488000 | 172.18.0.3 | 7311 | 127.0.0.1
READ_RSP message received from /172.18.0.3:7000 [Messaging-EventLoop-3-3] | 2026-09-20 23:07:33.489000 | 172.18.0.2 | 44 | 127.0.0.1
```

Q2 — Historial cronológico de un pasajero

Consulta con `WHERE pasajero_id = ? AND fecha_salida >= ? AND fecha_salida <= ? ORDER BY fecha_salida ASC`. Para el pasajero de prueba se obtuvieron 7 reservas, ordenadas cronológicamente, cada una con el estado del pago correspondiente (sin JOIN). El trace muestra `Skipped 0/1 non-slice-intersecting sstables`: el filtro de rango sobre la clustering key acota la lectura antes de tocar disco.

Q4 — Porcentaje de ocupación por ruta y rango de fechas

Ruta	Mes	Vuelo	Capacidad	Confirmadas	% Ocupación
MIA-LAX	2026-12	GT0476-20261204	120	82	68.33%

El porcentaje se calcula en la capa de aplicación ($\text{confirmadas} \times 100 / \text{capacidad}$) porque Cassandra no realiza divisiones entre columnas en el servidor; el bucketing (ruta, mes) evita que la partición de una ruta crezca sin límite con el paso de los años.

```
qlsh:aeorline> -- Q4
qlsh:aeorline> SELECT fecha_salida, codigo_vuelo, capacidad, confirmadas
... FROM ocupacion_por_ruta_mes
... WHERE ruta = 'MIA-LAX' AND mes = '2026-12'
... AND fecha_salida >= '2026-12-01 00:00:00+0000'
... AND fecha_salida <= '2026-12-31 23:59:59+0000';

fecha_salida | codigo_vuelo | capacidad | confirmadas
-----
2026-12-04 13:45:00-000000+0000 | GT0476-20261204 | 120 | 82
(1 row)

Tracing session: 10042108-b548-11f1-b091-530c2507f0f1

activity | source | source_elapsed | client | timestamp
-----
622000 | 172.18.0.2 | 0 | 127.0.0.1 | Execute CQL query | 2026-09-20 23:07:33.
622001 | 172.18.0.2 | 427 | 127.0.0.1 | Parsing SELECT fecha_salida, codigo_vuelo, capacidad, confirmadas FROM ocupacion_por_ruta_mes WHERE ruta = 'MIA-LAX' AND mes = '2026-12' AND fecha_salida >= '2026-12-01 00:00:00+0000' AND fecha_salida <= '2026-12-31 23:59:59+0000'; [Native-Transport-Requests-6] | 2026-09-20 23:07:33.
622002 | 172.18.0.2 | 725 | 127.0.0.1 | Preparing statement [Native-Transport-Requests-6] | 2026-09-20 23:07:33.
624000 | 172.18.0.2 | 2355 | 127.0.0.1 | reading data from /172.18.0.4:7000 [Native-Transport-Requests-6] | 2026-09-20 23:07:33.
624001 | 172.18.0.2 | 2063 | 127.0.0.1 | reading digest from /172.18.0.3:7000 [Native-Transport-Requests-6] | 2026-09-20 23:07:33.
624002 | 172.18.0.2 | 2742 | 127.0.0.1 | Sending READ_REQ message to /172.18.0.4:7000 message size 143 bytes [Messaging-EventLoop-3-9] | 2026-09-20 23:07:33.
625000 | 172.18.0.2 | 2800 | 127.0.0.1 | Sending READ_REQ message to /172.18.0.3:7000 message size 144 bytes [Messaging-EventLoop-3-2] | 2026-09-20 23:07:33.
625000 | 172.18.0.3 | 34 | 127.0.0.1 | READ_REQ message received from /172.18.0.2:7000 [Messaging-EventLoop-3-15] | 2026-09-20 23:07:33.
625000 | 172.18.0.4 | 31 | 127.0.0.1 | READ_REQ message received from /172.18.0.2:7000 [Messaging-EventLoop-3-14] | 2026-09-20 23:07:33.
625001 | 172.18.0.3 | 403 | 127.0.0.1 | Executing single-partition query on ocupacion_por_ruta_mes [ReadStage-1] | 2026-09-20 23:07:33.
625001 | 172.18.0.4 | 253 | 127.0.0.1 | Executing single-partition query on ocupacion_por_ruta_mes [ReadStage-5] | 2026-09-20 23:07:33.
625002 | 172.18.0.4 | 288 | 127.0.0.1 | Acquiring stable references [ReadStage-5] | 2026-09-20 23:07:33.
626000 | 172.18.0.3 | 508 | 127.0.0.1 | Acquiring stable references [ReadStage-1] | 2026-09-20 23:07:33.
626000 | 172.18.0.4 | 415 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-5] | 2026-09-20 23:07:33.
```

Q5 — Top N de vuelos por ingresos generados (octubre 2026, N=5)

#	Vuelo	Ruta	Ingresos
1	GT0686-20261027	MEX-SJO	\$56,961.48
2	GT0539-20261025	MIA-MEX	\$55,699.38
3	GT0462-20261003	MEX-SAL	\$55,628.09
4	GT0695-20261018	JFK-LAX	\$55,011.87
#	Vuelo	Ruta	Ingresos
5	GT0485-20261016	MIA-MAD	\$54,218.57

El trace muestra `Read 5 live rows`: Cassandra lee exactamente las 5 filas solicitadas porque la clustering key ya mantiene los vuelos ordenados por ingreso descendente (`CLUSTERING ORDER BY (total_centavos DESC)`); el `LIMIT 5` se resuelve leyendo directamente esas filas, sin ordenar ni filtrar en el cliente.

```

cqlsh:aerolineas> -- OS
cqlsh:aerolineas> SELECT codigo_vuelo, ruta, total_centavos
... FROM top_vuelos_por_ingresos
... WHERE periodo = '2026-10'
... LIMIT 5;

codigo_vuelo | ruta | total_centavos
-----
GT0686-20261027 | MEX-SJO | 5696148
GT0539-20261025 | MIA-MEX | 5569938
GT0462-20261003 | MEX-SAL | 5562000
GT0695-20261018 | JFK-LAX | 5500187
GT0485-20261016 | MIA-PMD | 5421857

(5 rows)

Tracing session: 361bfdb0-b549-11f1-ba91-538c2587f0f1

activity | timestamp | source | source_elapsed | client
-----
Parsing SELECT codigo_vuelo, ruta, total_centavos FROM top_vuelos_por_ingresos WHERE periodo = '2026-10' LIMIT 5; [Native-Transport-Requests-1] 2026-09-20 23:15:47.024000 172.18.0.2 0 127.0.0.1
Preparing statement [Native-Transport-Requests-1] 2026-09-20 23:15:47.025000 172.18.0.2 326 127.0.0.1
reading digest from /172.18.0.3:7000 [Native-Transport-Requests-1] 2026-09-20 23:15:47.025001 172.18.0.2 689 127.0.0.1
Executing single-partition query on top_vuelos_por_ingresos [ReadStage-2] 2026-09-20 23:15:47.025002 172.18.0.2 1147 127.0.0.1
Acquiring stable references [ReadStage-2] 2026-09-20 23:15:47.025003 172.18.0.2 1151 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-2] 2026-09-20 23:15:47.025004 172.18.0.2 1191 127.0.0.1
speculating read retry on Full(/172.18.0.4:7000, (-2627662688637150199, -234606895195170617)) [Native-Transport-Requests-1] 2026-09-20 23:15:47.025005 172.18.0.2 1240 127.0.0.1
Sending READ_REQ message to /172.18.0.3:7000 message size 99 bytes [Messaging-EventLoop-3-2] 2026-09-20 23:15:47.025006 172.18.0.2 1291 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-2] 2026-09-20 23:15:47.026000 172.18.0.2 1335 127.0.0.1
READ_REQ message received from /172.18.0.2:7000 [Messaging-EventLoop-3-14] 2026-09-20 23:15:47.026000 172.18.0.2 1435 127.0.0.1
Sending READ_REQ message to /172.18.0.4:7000 message size 98 bytes [Messaging-EventLoop-3-9] 2026-09-20 23:15:47.026001 172.18.0.2 96 127.0.0.1
Read 5 live rows and 0 tombstone cells [ReadStage-2] 2026-09-20 23:15:47.026002 172.18.0.2 74 127.0.0.1
Executing single-partition query on top_vuelos_por_ingresos [ReadStage-3] 2026-09-20 23:15:47.027000 172.18.0.3 1464 127.0.0.1
Acquiring stable references [ReadStage-3] 2026-09-20 23:15:47.027001 172.18.0.3 1497 127.0.0.1
Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-3] 2026-09-20 23:15:47.027002 172.18.0.3 530 127.0.0.1
Merged data from memtables and 0 sstables [ReadStage-3] 2026-09-20 23:15:47.027003 172.18.0.3 513 127.0.0.1
Read 5 live rows and 0 tombstone cells [ReadStage-3] 2026-09-20 23:15:47.027004 172.18.0.3 738 127.0.0.1
Enqueuing response to /172.18.0.2:7000 [ReadStage-1] 2026-09-20 23:15:47.027005 172.18.0.4 574 127.0.0.1
Sending READ_RSP message to cassi/172.18.0.2:7000 message size 253 bytes [Messaging-EventLoop-3-1] 2026-09-20 23:15:47.027006 172.18.0.4 837 127.0.0.1
READ_RSP message received from /172.18.0.4:7000 [Messaging-EventLoop-3-4] 2026-09-20 23:15:47.028000 172.18.0.2 714 127.0.0.1

```

7.1 Latencias medidas

```

root@5376f7b73087:/app# python python/consultas.py
Consistency Level: QUORUM | repeticiones: 100 | vuelo: GT0122-20261021 | ruta: GUA-MEX

| Consulta | filas | promedio (ms) | p50 (ms) | p95 (ms) | max (ms) |
|---|---|---|---|---|---|
| Q1 Disponibilidad por clase | 3 | 3.20 | 3.17 | 4.14 | 6.96 |
| Q2 Historial de pasajero | 7 | 4.16 | 4.09 | 5.04 | 7.64 |
| Q3 Manifiesto de vuelo | 146 | 4.13 | 3.94 | 5.83 | 10.77 |
| Q4 % ocupacion ruta/fechas | 3 | 3.15 | 3.05 | 4.08 | 8.05 |
| Q5 Top N ingresos | 5 | 8.13 | 8.03 | 9.72 | 16.26 |

--- Resultados de ejemplo (primeras filas) ---

Q1 Disponibilidad por clase
('economica', 29, 115)
('ejecutiva', 5, 19)
('primera', 0, 12)

Q2 Historial de pasajero
(datetime.datetime(2026, 10, 4, 21, 30), 'GT0391-20261004', 'JFK', 'MIA', 2, 'C', 'primera', 'confirmada', Decimal('1302.47'), 'pagado')
(datetime.datetime(2026, 12, 26, 12, 45), 'GT0731-20261226', 'SJO', 'GUA', 2, 'A', 'primera', 'confirmada', Decimal('1623.14'), 'pagado')
(datetime.datetime(2026, 12, 30, 13, 0), 'GT0287-20261230', 'BOG', 'GUA', 1, 'B', 'primera', 'confirmada', Decimal('2099.23'), 'pagado')
(datetime.datetime(2026, 12, 31, 13, 30), 'GT0502-20261231', 'MIA', 'MAD', 18, 'E', 'economica', 'confirmada', Decimal('303.76'), 'pagado')
(datetime.datetime(2027, 1, 7, 10, 15), 'GT0184-20270107', 'MEX', 'JFK', 21, 'E', 'economica', 'confirmada', Decimal('187.44'), 'pagado')

Q3 Manifiesto de vuelo
(1, 'A', 'Ramón del Rivero', '1971399378410', 'primera', 'confirmada', 'pagado')
(1, 'B', 'Ricardo de Barros', '6962775459510', 'primera', 'pendiente', 'pendiente')
(1, 'C', 'Loreto Cid Cordero', '4971519542875', 'primera', 'confirmada', 'pagado')
(1, 'D', 'Emma Bravo-Tejada', '6536265199598', 'primera', 'confirmada', 'pagado')
(1, 'E', 'Berto Guardia Sandoval', '3572129666363', 'primera', 'confirmada', 'pagado')

Q4 % ocupacion ruta/fechas
('GT0474-20261012', 120, 78, 65.0)
('GT0122-20261021', 180, 131, 72.78)
('GT0361-20261028', 120, 91, 75.83)

Q5 Top N ingresos
('GT0331-20261104', 'BOG-SAL', 58983.84)
('GT0686-20261027', 'MEX-SJO', 56961.48)
('GT0658-20261231', 'LAX-GUA', 56892.59)
('GT0539-20261025', 'MIA-MEX', 55699.38)
('GT0462-20261003', 'MEX-SAL', 55628.09)
root@5376f7b73087:/app#

```

8. Reglas de negocio en un entorno distribuido

8.1 Un asiento no puede ser ocupado por dos pasajeros (LWT)

Cassandra no ofrece bloqueos ni transacciones ACID multi-fila. Para garantizar que un mismo asiento no quede asignado a dos reservas al mismo tiempo, se usa una Lightweight Transaction (Paxos) sobre `asientos_por_vuelo`:

```
UPDATE asientos_por_vuelo SET estado = 'ocupado', reserva_id = ?
WHERE codigo_vuelo = ? AND fila = ? AND letra = ?
IF estado = 'disponible';
```

Prueba: se simularon dos pasajeros (A y B) solicitando el mismo asiento en el mismo instante. El pasajero A obtuvo `[applied]=True` y quedó confirmado; el pasajero B obtuvo `[applied]=False` y fue rechazado. El estado final del asiento confirmó que solo la reserva de A quedó registrada, cumpliendo la regla de negocio 2 del enunciado.

```
root@5376f7b73087:/app# python python/reglas_negocio.py --demo-lwt --demo-ttl
Regla 2: un asiento no puede quedar en manos de dos pasajeros

Asiento de prueba: GT0122-20261021 fila 99 letra Z -> disponible

Pasajero A intenta tomar el asiento (reserva 808df1fa-cf7e-41b3-ab7a-fd5bf85b5ac0)
-> CONFIRMADO
Pasajero B intenta tomar el MISMO asiento (reserva 8016df53-9032-4a9e-9a35-78ca05409d68)
-> RECHAZADO (alguien mas lo tomo)

Estado final del asiento: ocupado, reserva_id = 808df1fa-cf7e-41b3-ab7a-fd5bf85b5ac0
OK: solo una de las dos reservas tuvo exito; el asiento quedo con la reserva de A.

TTL: una reserva pendiente que caduca si no se confirma a tiempo

(TTL de demostracion: 8 segundos; en produccion serian 900 = 15 min)

Traceback (most recent call last):
  File "/app/python/reglas_negocio.py", line 166, in <module>
    main()
  File "/app/python/reglas_negocio.py", line 160, in main
    demo_ttl(s, codigo)
  File "/app/python/reglas_negocio.py", line 114, in demo_ttl
    session.execute(
  File "cassandra/cluster.py", line 2642, in cassandra.cluster.Session.execute
  File "cassandra/cluster.py", line 2685, in cassandra.cluster.Session.execute_async
  File "cassandra/cluster.py", line 3036, in cassandra.cluster.Session._create_response_future
  File "cassandra/query.py", line 859, in cassandra.query.bind_params
TypeError: not enough arguments for format string
```

8.2 Expiración automática de reservas pendientes (TTL)

Una reserva en estado *pendiente* que no se confirma a tiempo debe liberar el asiento automáticamente. Se escribe con `USING TTL` (900 segundos = 15 minutos en producción):

```
INSERT INTO reservas_por_id (reserva_id, pasajero_id, ...)
VALUES (?, ?, ...)
USING TTL 900;
```

Prueba: con fines de demostración se usó un TTL de 8 segundos. Tras crear la reserva, una consulta inmediata mostró `TTL(estado) = 8`; al reconsultar 10 segundos después, la fila ya no existía (Cassandra la eliminó automáticamente vía tombstone). Cassandra no libera el asiento asociado por sí sola: en producción, un proceso periódico debe revisar los asientos en estado *bloqueado* sin una reserva viva y devolverlos a *disponible*, o bien resolverlo de forma perezosa en el momento en que la aplicación muestra la disponibilidad del vuelo.

```
TypeError: not enough arguments for format string
root@5376f7b73887:/app# python python/reglas_negocio.py --demo-ttl
TTL: una reserva pendiente que caduca si no se confirma a tiempo

(TTL de demostracion: 8 segundos; en produccion serian 900 = 15 min)

Reserva 73c90f6b-7ba8-4873-b809-52aa51bd85e creada como 'pendiente' con TTL.
Recien creada -> estado=pendiente, TTL restante=8 s
Esperando 10 s a que el TTL expire...
Tras esperar -> la fila ya NO existe (expiro sola)
OK: la reserva pendiente desaparecio sola por TTL, sin borrarla a mano.
Asiento fila 98 letra Y liberado manualmente (en produccion: un job periodico
revisa asientos 'bloqueado' sin reserva viva y los vuelve a 'disponible').
root@5376f7b73887:/app# python python/consultas.py
Consistency Level: QUORUM | repeticiones: 100 | vuelo: GT0122-20261021 | ruta: GUA-MEX

| Consulta | filas | promedio (ms) | p50 (ms) | p95 (ms) | max (ms) |
|---|---|---|---|---|---|
| Q1 Disponibilidad por clase | 3 | 3.20 | 3.17 | 4.14 | 6.96 |
| Q2 Historial de pasajero | 7 | 4.16 | 4.09 | 5.04 | 7.64 |
| Q3 Manifiesto de vuelo | 146 | 4.13 | 3.94 | 5.83 | 10.77 |
| Q4 % ocupacion ruta/fechas | 3 | 3.15 | 3.05 | 4.08 | 8.05 |
| Q5 Top N ingresos | 5 | 8.13 | 8.03 | 9.72 | 16.26 |

--- Resultados de ejemplo (primeras filas) ---

Q1 Disponibilidad por clase
('economica', 29, 115)
('ejecutiva', 5, 19)
```

9. Pruebas de escalabilidad y tolerancia a fallos

Se midieron 100 lecturas (Q3, manifiesto del vuelo GT0476-20261204) y 100 escrituras (sobre una tabla auxiliar) por cada Consistency Level, en tres escenarios: los 3 nodos activos, 1 nodo caído (cass3 detenido) y 2 nodos caídos (cass2 y cass3 detenidos).

```
Try Docker debug for seamless, persistent debugging tools in any container or image -> docker debug cass1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker stop cass3
cass3
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
// State=Normal/Leaving/Joining/Moving
-- Address Load Tokens Owns (effective) Host ID Rack
DN 172.18.0.4 34.64 MiB 16 100.0% 14873b2e-ec80-46b2-9e5d-d7b731000845 rack1
UN 172.18.0.3 34.65 MiB 16 100.0% 9dbe660f-68b9-488c-84ce-f0525e13cfb5 rack1
UN 172.18.0.2 34.6 MiB 16 100.0% 69cb9021-8ddd-4c9b-8cf2-539168bcac19 rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> |
```

9.1 Disponibilidad por escenario

Escenario	CL	Operación	Éxitos	% éxito	Promedio (ms)	p95 (ms)
3 nodos activos	ONE	lectura	100/100	100%	1.81	2.59
3 nodos activos	ONE	escritura	100/100	100%	1.17	1.46
3 nodos activos	QUORUM	lectura	100/100	100%	2.82	3.80
3 nodos activos	QUORUM	escritura	100/100	100%	1.09	1.46
3 nodos activos	ALL	lectura	100/100	100%	2.75	4.21
Escenario	CL	Operación	Éxitos	% éxito	Promedio (ms)	p95 (ms)

3 nodos activos	ALL	escritura	100/100	100%	1.30	1.78
1 nodo caído	ONE	lectura	100/100	100%	1.59	2.22
1 nodo caído	ONE	escritura	100/100	100%	1.05	1.81
1 nodo caído	QUORUM	lectura	100/100	100%	2.58	3.52
1 nodo caído	QUORUM	escritura	100/100	100%	1.47	2.09
1 nodo caído	ALL	lectura	0/100	0%	—	—
1 nodo caído	ALL	escritura	0/100	0%	—	—
2 nodos caídos	ONE	lectura	100/100	100%	1.42	2.05
2 nodos caídos	ONE	escritura	100/100	100%	0.85	1.22
2 nodos caídos	QUORUM	lectura	0/100	0%	—	—
2 nodos caídos	QUORUM	escritura	0/100	0%	—	—
2 nodos caídos	ALL	lectura	0/100	0%	—	—
2 nodos caídos	ALL	escritura	0/100	0%	—	—

Con RF=3, cada Consistency Level tolera hasta (3 – réplicas requeridas) nodos caídos: ONE requiere 1 réplica y tolera 2 nodos caídos; QUORUM requiere 2 réplicas y tolera 1 nodo caído; ALL requiere las 3 réplicas y no tolera ninguno. Los resultados obtenidos coinciden exactamente con esta predicción teórica: en cada caso donde el número de réplicas vivas cayó por debajo de lo requerido, las operaciones fallaron de inmediato con `Unavailable` — no con `timeout` — porque el nodo coordinador detecta antes de intentar que no hay réplicas suficientes.

9.2 Rendimiento (3 nodos activos)

Las lecturas con ONE fueron las más rápidas (1.81 ms) frente a QUORUM y ALL (~2.8 ms), porque con ONE el coordinador solo espera la respuesta de una réplica; con QUORUM y ALL debe además comparar un *digest* de otra(s) réplica(s) para garantizar consistencia. Las escrituras, en cambio, apenas variaron entre niveles (1.09–1.30 ms): Cassandra siempre envía la escritura a todas las réplicas vivas, y el Consistency Level solo decide cuántas confirmaciones debe esperar el coordinador antes de responder al cliente. La diferencia entre QUORUM y ALL en lectura fue menor a 0.1 ms en promedio (aunque el p95 de ALL sí fue mayor: 4.21 ms frente a 3.80 ms), lo que es esperable en un clúster de laboratorio con los 3 nodos en la misma máquina; en un despliegue con nodos en ubicaciones distintas la brecha entre ONE/QUORUM/ALL sería más notoria debido a la latencia de red.

```

| Escenario | CL escritura | Escrituras exitosas | Filas leídas con ALL | Estado |
|---|---|---|---|---|
| 1_nodo_caído | ALL | 0 | 0 | OK |
| 1_nodo_caído | ONE | 100 | 100 | OK |
| 1_nodo_caído | QUORUM | 100 | 100 | OK |
| 2_nodos_caídos | ALL | 0 | 0 | OK |
| 2_nodos_caídos | ONE | 100 | 100 | OK |
| 2_nodos_caídos | QUORUM | 0 | 0 | OK |
| 3_nodos | ALL | 100 | 100 | OK |
| 3_nodos | ONE | 100 | 100 | OK |
| 3_nodos | QUORUM | 100 | 100 | OK |
root@171c27e71b9a:/app# python python/pruebas_consistencia.py --resumen
| Escenario | CL | Operacion | Exitos | % exito | Prom (ms) | p50 (ms) | p95 (ms) | Errores |
|---|---|---|---|---|---|---|---|---|
| 3_nodos | ONE | lectura | 100/100 | 100% | 1.81 | 1.74 | 2.59 | - |
| 3_nodos | ONE | escritura | 100/100 | 100% | 1.17 | 1.13 | 1.46 | - |
| 3_nodos | QUORUM | lectura | 100/100 | 100% | 2.82 | 2.77 | 3.80 | - |
| 3_nodos | QUORUM | escritura | 100/100 | 100% | 1.09 | 1.03 | 1.46 | - |
| 3_nodos | ALL | lectura | 100/100 | 100% | 2.75 | 2.48 | 4.21 | - |
| 3_nodos | ALL | escritura | 100/100 | 100% | 1.30 | 1.24 | 1.78 | - |
| 1_nodo_caído | ONE | lectura | 100/100 | 100% | 1.59 | 1.51 | 2.22 | - |
| 1_nodo_caído | ONE | escritura | 100/100 | 100% | 1.05 | 0.95 | 1.81 | - |
| 1_nodo_caído | QUORUM | lectura | 100/100 | 100% | 2.58 | 2.46 | 3.52 | - |
| 1_nodo_caído | QUORUM | escritura | 100/100 | 100% | 1.47 | 1.44 | 2.09 | - |
| 1_nodo_caído | ALL | lectura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 1_nodo_caído | ALL | escritura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | ONE | lectura | 100/100 | 100% | 1.42 | 1.34 | 2.05 | - |
| 2_nodos_caídos | ONE | escritura | 100/100 | 100% | 0.85 | 0.70 | 1.22 | - |
| 2_nodos_caídos | QUORUM | lectura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | QUORUM | escritura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | ALL | lectura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | ALL | escritura | 0/100 | 0% | - | - | - | Unavailable=100 |
root@171c27e71b9a:/app#

```

9.3 Recuperación tras restaurar los nodos

Al reiniciar los nodos caídos (`docker start cass2 cass3`) y confirmar con `nodetool status` que los 3 volvían a estado UN, se ejecutó `nodetool repair aerolinea` y se leyó con CL=ALL cada conjunto de datos escrito durante los escenarios de falla. Las 100 escrituras realizadas con CL=ONE mientras había 2 nodos caídos se recuperaron completas: las 3 réplicas coincidían tras la reparación, gracias a los *hinted handoffs* del coordinador y al repair posterior. Esto confirma que ninguna escritura aceptada por Cassandra se perdió al restaurar el clúster, aun cuando temporalmente solo existía 1 réplica disponible.

9.4 Recomendación de Consistency Level para producción

Para **escrituras de reservas y pagos** se recomienda **QUORUM**: tolera un nodo caído sin sacrificar consistencia fuerte cuando también se lee con QUORUM ($R + W = 2 + 2 = 4 > RF = 3$). Para **lecturas de bajo riesgo**, como mostrar el manifiesto de un vuelo a un agente de soporte, **ONE** ofrece la mejor disponibilidad y latencia, aceptando el riesgo de leer un dato ligeramente desactualizado en el peor caso.

10. Conclusiones

- El modelado basado en queries permitió responder las 5 consultas de negocio leyendo siempre una sola partición, sin JOINS ni ALLOW FILTERING, a costa de denormalizar los datos en varias tablas que deben mantenerse sincronizadas desde la capa de aplicación (batches logged y contadores).
- Las pruebas de tolerancia a fallos confirmaron de forma empírica la relación teórica entre Replication Factor, Consistency Level y disponibilidad: el sistema siguió respondiendo con hasta 2 nodos caídos usando CL=ONE, y se degradó exactamente como predice la fórmula de quórum al usar QUORUM o ALL.
- Las Lightweight Transactions resolvieron la única regla de negocio que requiere consistencia fuerte a nivel de fila (un asiento, un pasajero), a costa de la latencia adicional del protocolo Paxos, por lo que se reservan solo para esa operación puntual y no para el resto del sistema.